DATA ANALYSIS USING ROOT

Presented by,
Riya Kumari Karn
Enrollment No: CUSB2404112028
M.Sc. Physics (4th sem)

- **Basics of ROOT (done):-**
  → Creating a graph, histogram, etc.
  → Adding error bars
  → Curve fitting
  → Statistical analysis (mean, standard deviation, etc.)
  → Energy Calibration
  → Energy Resolution

- **Basics of HPGe (done):-** Studied about (regarding HPGe)-
  → Working
  → Principle
  → Operational Characteristics

Why Data Analysis Is Required in Physics?

Physics is an experimental science, and raw data by itself has no meaning until it is properly analyzed. Data analysis is the bridge between measurement and physical understanding.

1. Converting Raw Data into Physical Meaning

Experiments produce raw numbers:

- Counts
- Voltages
- Time stamps
- Channel numbers

These quantities do not directly represent physical laws.

Data analysis is required to convert them into:

- Energy
- Intensity
- Resolution
- Lifetime
- Probability

Example:

HPGe detector gives channel number vs counts → data analysis converts this into energy spectrum and identifies gamma-ray energies.

2. Extracting Physical Parameters

Data analysis allows physicists to determine important physical quantities such as:

- Mean values
- Peak positions
- Widths (FWHM)
- Energy resolution
- Decay constants
- Cross sections

Example:

Gaussian fitting of a spectral peak gives:

- Mean → photon energy
- σ → detector resolution
- FWHM → energy spread

Without data analysis, these quantities cannot be measured accurately.

INTRODUCTION:-

ROOT is an object-oriented, open-source software framework developed at CERN for data processing, analysis, visualization, and storage, especially in high-energy physics (HEP) and nuclear/astroparticle physics. It is widely used in experiments where very large volumes of data must be handled efficiently.
HPGe detector data analysis in ROOT involves histogramming of gamma spectra, energy calibration using standard sources, Gaussian peak fitting, energy calibration.

Key Features of ROOT:-

(a) Data Storage-

- ROOT uses a highly efficient binary file format called .root
- Data are stored in structures like:
  TTree – for large structured datasets
  TNtuple – simplified data container
- Supports data compression and fast I/O

(b) Data Analysis-

ROOT provides powerful tools for:

- Statistical analysis
- Curve fitting
- Parameter estimation
- Hypothesis testing
- TH1, TH2, TH3 – 1D, 2D, and 3D histograms
- TF1, TF2 – functions for fitting
- TGraph, TGraphErrors – graphs with or without error bars

(c) Curve Fitting

ROOT supports:

- Gaussian, polynomial, exponential, etc.
- User-defined functions
- Chi-square

Example applications:

- Energy calibration
- Peak identification
- Background subtraction

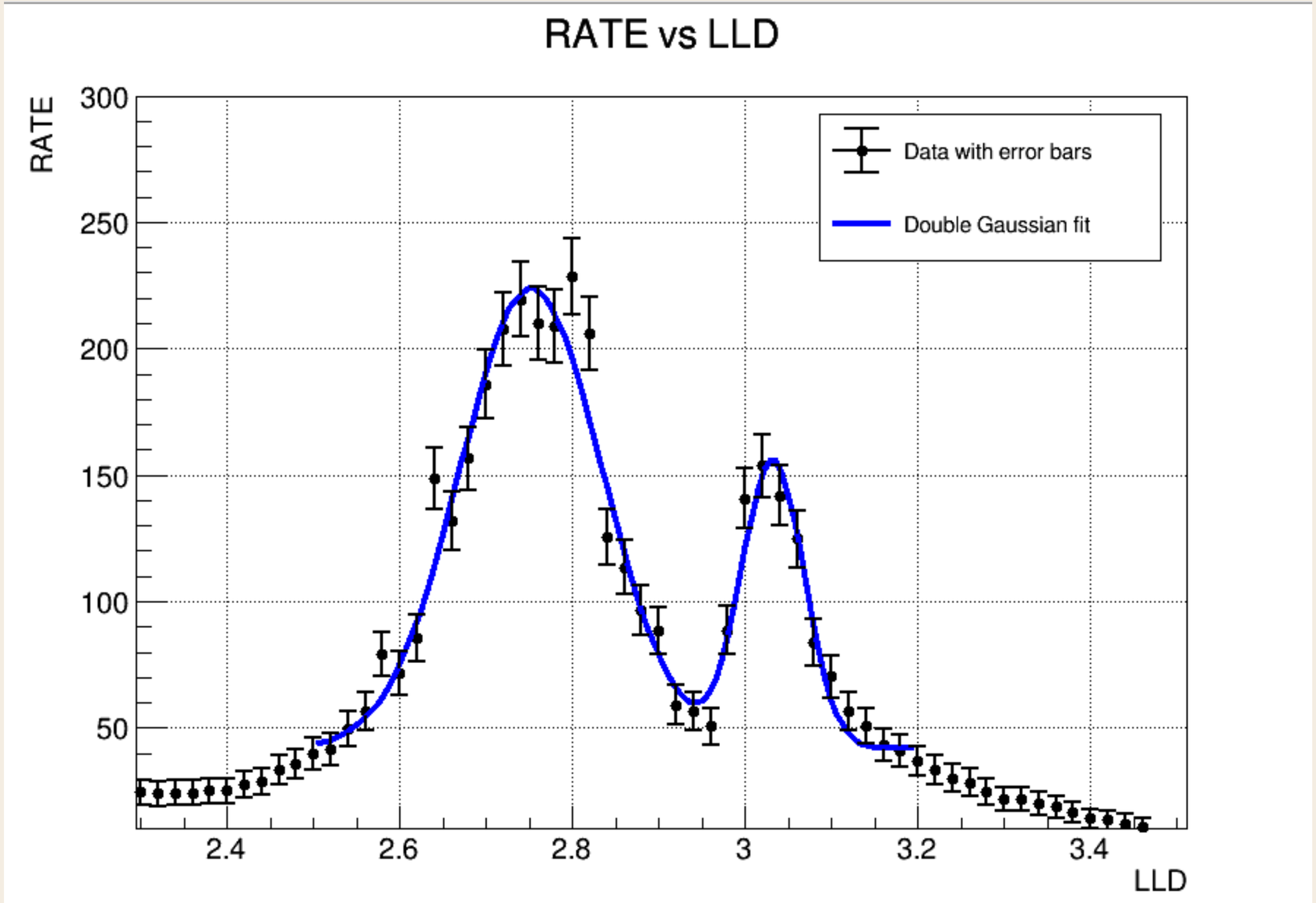
(d) Data Visualization

ROOT can create:

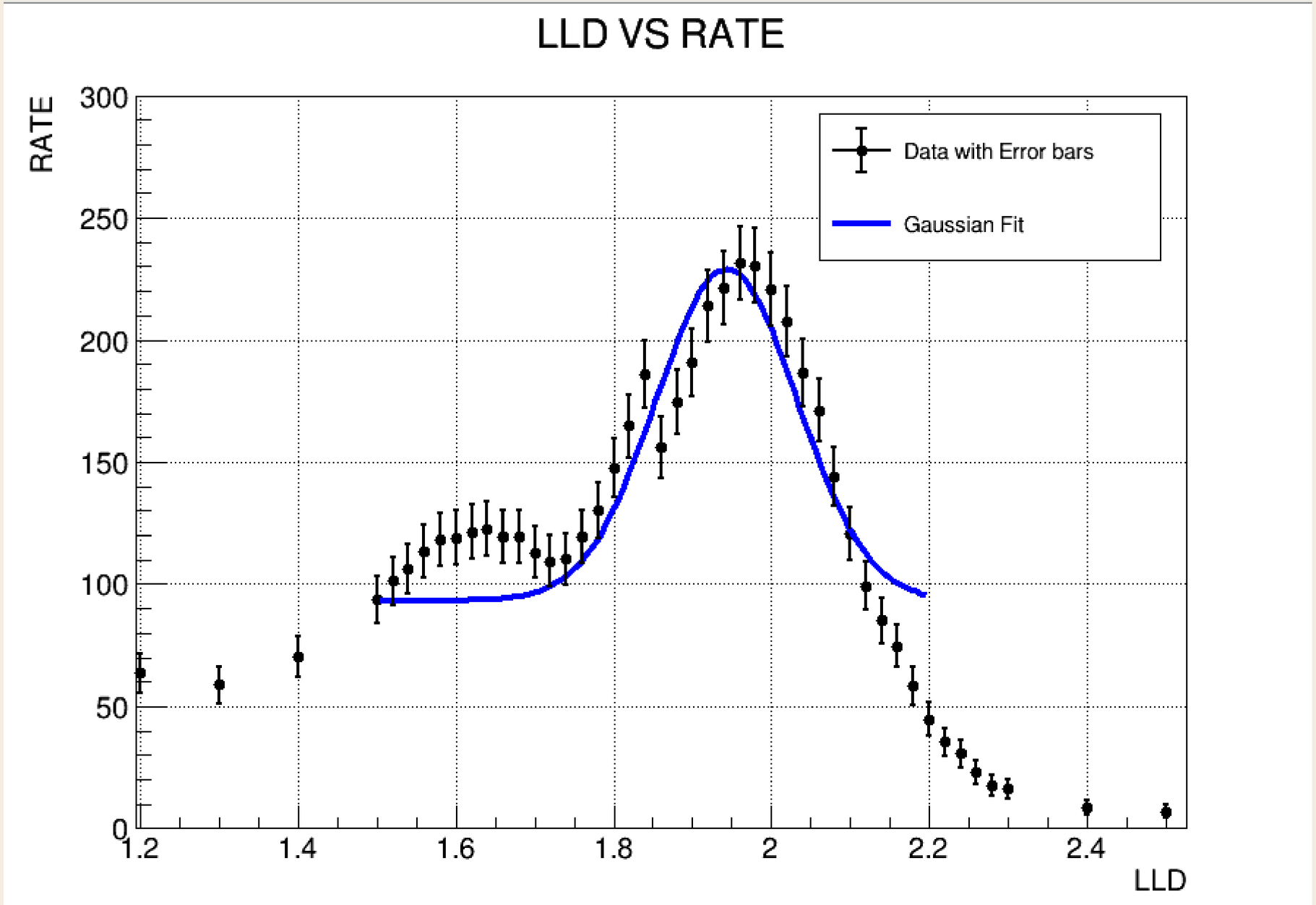
- Histograms and scatter plots
- Log/linear scale graphs
- Publication-ready figures
- TCanvas – drawing area
- TPad – subdivisions of canvas
- TLegend, TLatex – annotations and equations

Energy Calibration using ROOT:

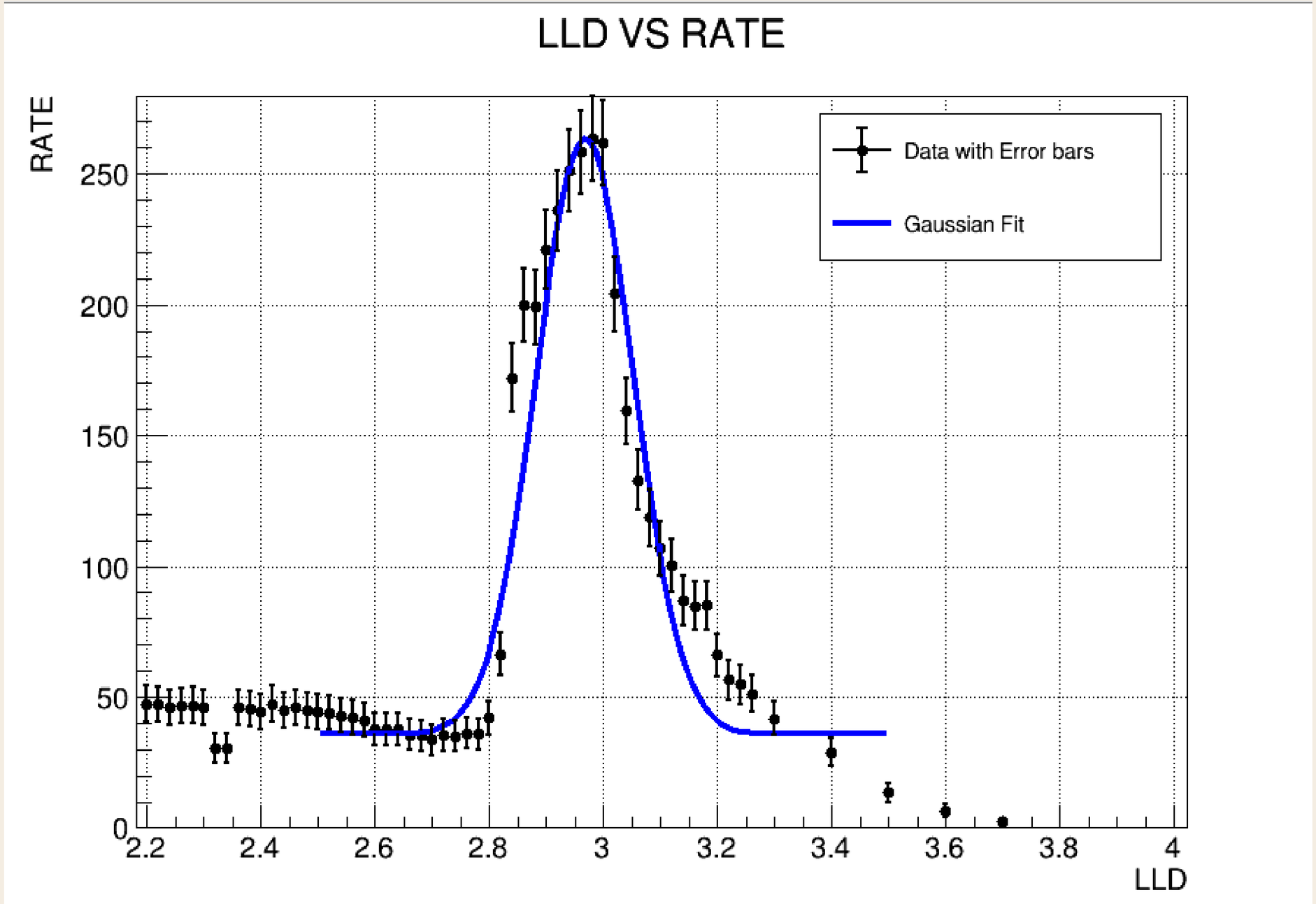
- Sodium Plot:



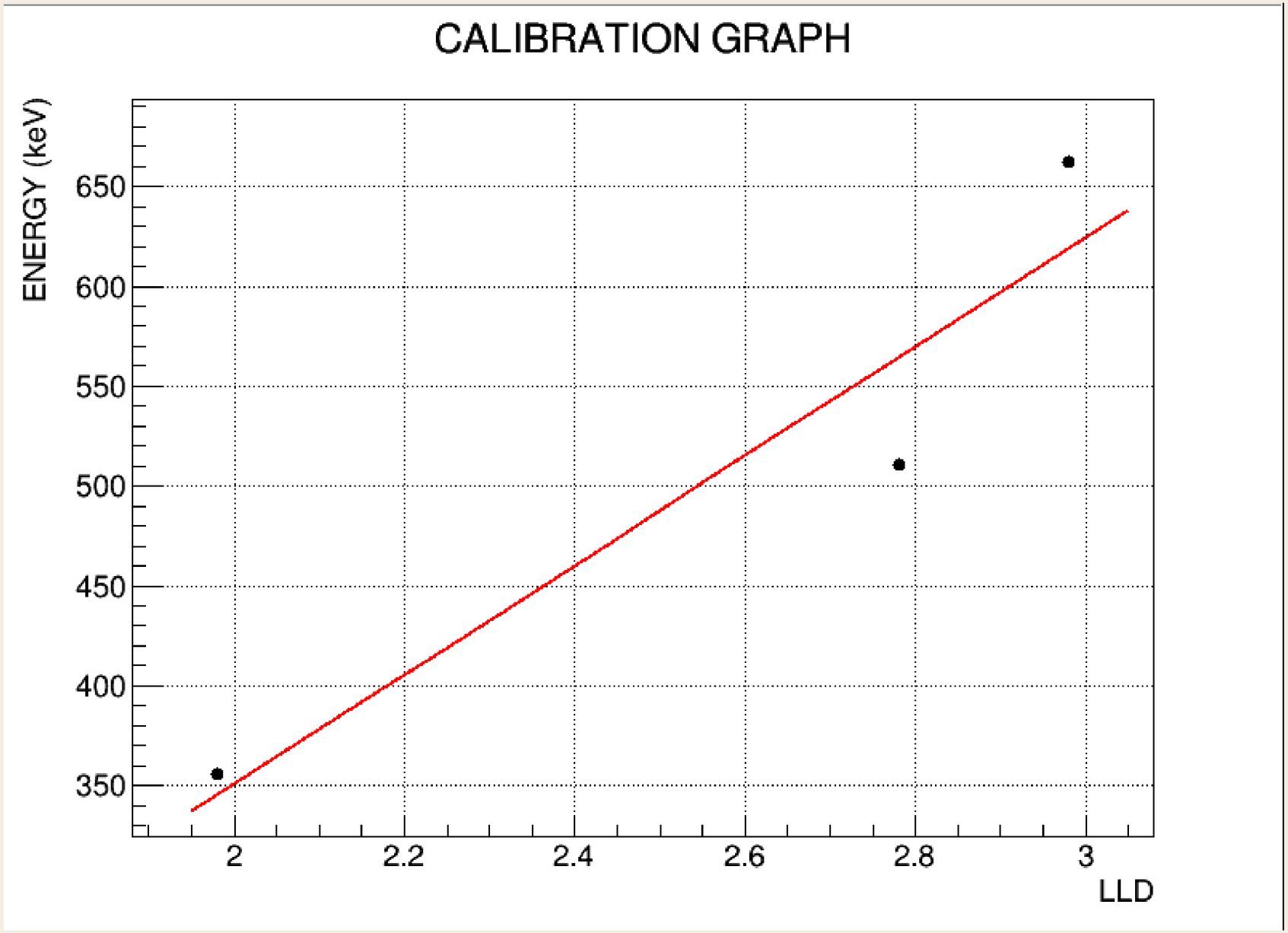
- **Barium Plot:-**


LLD VS RATE
RATE
300
250
200
150
100
50
0
1.2
1.4
1.6
1.8
2
2.2
2.4
LLD
Data with Error bars
Gaussian Fit

- **Caesium Plot:**



• Calibration Curve:-



a= -197.069 +/- 3.49568
b= 273.929 +/- 1.33631
Calibration equation:
E = (-197.069 ± 3.49568) + (273.929 ± 1.33631) * LLD

Code Description:

1. Function definition:-

```
void bariumcode()
{
```

Defines a ROOT macro function named bariumcode. When we type bariumcode(); in ROOT, this function will execute.

2. Reading data from file:-

```
ifstream file("barium3.txt");
```

Opens the text file barium3.txt. The file is expected to contain two columns:
LLD RATE

3. Declaring vectors:-

```
vector<double> lld, rate;
vector<double> errX, errY;
```

lld → stores Lower Level Discriminator values (x-axis)
rate → stores count rate (y-axis)
errX → error in LLD (x-error)
errY → error in RATE (y-error)

4. Reading data line by line:-

```
double x, y;
while (file >> x >> y) {
    lld.push_back(x);
    rate.push_back(y);
    errX.push_back(0.0);
    errY.push_back(sqrt(y));
}
```

Reads data until the file ends.
x → LLD value
y → RATE value
Stores: LLD values in lld, RATE values in rate
Error assignment:
errX = 0.0 → LLD uncertainty neglected
errY = $\sqrt{y}$ → Poisson statistics ; Counting errors in radiation detection are statistical → $\sigma = \sqrt{N}$

5. Close file:-

```
file.close();
```

Closes the file safely after reading.

6. Number of data points:-

```
int n = lld.size();
```

Stores total number of data points. Needed for graph creation.

7. Creating TGraphErrors:-

```
TGraphErrors *gr = new TGraphErrors(
 n,
 &lld[0], &rate[0],
 &errX[0], &errY[0]
);
```

Creates a graph with error bars. Arguments:
n → number of points
lld → x values
rate → y values
errX → x errors
errY → y errors
Used instead of TGraph because error bars are needed.

8. Graph styling:-

```
gr->SetTitle("LLD VS RATE;LLD;RATE");
```

Sets:
Plot title: LLD VS RATE
X-axis label: LLD
Y-axis label: RATE

```
gr->SetMarkerStyle(20);
```
(Circular filled markers)

```
gr->SetMarkerSize(0.9);
```
(Marker size adjusted)

```
gr->SetLineWidth(2);
```
(Thicker lines for visibility)

9. Defining Gaussian + background fit:-

```
TF1 *fit = new TF1(
 "fit",
 "[0]*exp(-0.5*((x-[1])*(x-[1])/([2]*[2]))) + [3]",
 1.5, 2.2
);
```

Fit function meaning:

$$f(x) = A \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) + B$$

Parameter Meaning
[0] Amplitude (peak height)
`[1] Mean (peak position)
[2] Sigma (peak width)
[3] Constant background
Fit range: 1.5 to 2.2 LLD
Background included because radiation spectra always have noise.

10. Initial fit parameters:-

```
fit->SetParameters(280, 1.98, 0.08, 30);
```

Peak height ≈ 280
Peak position ≈ 1.98
Width ≈ 0.08
Background ≈ 30

11. Fit styling:-

```
fit->SetLineColor(kBlue);
fit->SetLineWidth(4);
```

Blue thick curve for fit function.

12. Canvas creation:-

```
TCanvas *c1 = new TCanvas("c1", "Gaussian Fit", 900, 650);
```

Window size: 900 × 650 pixels
Title: Gaussian Fit

13. Error bar cap size:-
gStyle->SetEndErrorSize(2);
Adjusting error bar “T-caps” size.

14. Drawing graph:-
gr->Draw("APE");
"A" → draw axes
"P" → draw points
"E" → draw error bars

15. Axis range settings:-
gr->GetXaxis()->SetRangeUser(1.2, 2.5);
gr->GetYaxis()->SetRangeUser(0, 300);
Zooms into the region of interest. Avoids unnecessary empty regions.

16. Performing the fit:-
gr->Fit(fit, "R");
Fits only within the function range (1.5–2.2)
"R" → restricts fit to defined limits

17. Grid:-
gPad->SetGrid();
Adds grid lines for easier reading.

18. Legend:-
TLegend *leg = new TLegend(0.62, 0.72, 0.88, 0.88);
Defines legend position.
leg->AddEntry(gr, "Data with Error bars", "lep");
leg->AddEntry(fit, "Gaussian Fit", "l");
leg->Draw();
Explains plot elements clearly.

Energy Calibration:

Energy calibration is a fundamental step in radiation detector data analysis. It establishes a quantitative relationship between the detector's raw output (such as channel number, ADC value, or LLD) and the actual energy of incident radiation.

Principle of Energy Calibration:

For most detectors (especially HPGe and NaI(Tl)), the detector response is approximately linear over a wide energy range. Hence, the relation between energy and channel number is given by:

$$E=a+bC$$; where:

- E = Energy (keV or MeV)
- C = Channel number
- a = Offset/intercept (zero-energy intercept)
- b = Gain/slope (energy per channel)

Energy calibration allows us to:

- Convert channel numbers into physical units (keV or MeV)
- Measure peak positions accurately
- Determine energy resolution (FWHM/E)
- Compare experimental spectra with reference data